🎬 Shockwave — demo recording guide (≤ 3:00)

Read the **bold VO** lines aloud; do the `[ON SCREEN]` actions. Target ~2:50. **Everything records from the live site — no local setup needed.**

Pre-flight

Open these browser tabs, in order: 1. **Live site:** `https://shockwave-ochw.vercel.app/` 2. **Blast Monitor:** `https://shockwave-ochw.vercel.app/app` 3. **Live MR-bot comment:** `https://gitlab.com/uthmannabeel-group/Shockwave-project/-/merge_requests/1` 4. **AI Catalog agent:** `https://gitlab.com/explore/ai-catalog/agents/1011457/`

Recording setup: 1080p, browser zoom ~110%, hide the bookmarks bar. Tools: OBS / Loom / Xbox Game Bar (`Win+G`).

The script

0:00 – 0:15 · Hook

`[ON SCREEN]` Tab 1 (landing), scroll slowly.

> **"You're reviewing a one-line change. What does it break? `grep` can't tell you — it can't follow a cross-file call graph. But GitLab Orbit already built one. This is Shockwave."**

0:15 – 1:10 · The Blast Monitor (the hero)

`[ON SCREEN]` Click **Launch Blast Monitor** → type `setupmethod` → **Detonate**.

> **"I'm changing `setupmethod` — a tiny internal decorator in Flask. Watch the blast radius ripple out: 114 definitions across 10 files depend on it."**

`[ON SCREEN]` Point to the **HIGH RISK 100/100** badge, then the cyan nodes.

> **"Shockwave gives a verdict — HIGH risk — and shows it's reachable from a public entry point: Flask's `@route` decorator. A break here is externally observable, not internal."**

`[ON SCREEN]` Scroll the right rail to **Tests to run**.

> **"And instead of running the whole suite, it tells me the 58 tests that actually exercise this change — copy, paste, run. Plus a generated stub for the hotspots nothing covers."**

1:10 – 1:30 · It's the real graph

`[ON SCREEN]` Toggle light/dark with the sun/moon icon; click `route` or `dispatch_request` to re-detonate.

> **"This isn't a mock — it's a real analysis from Orbit's graph. The same engine runs in your terminal, on Orbit Local *and* the hosted Orbit Remote over its API."**

1:30 – 2:10 · The autonomous reviewer (the payoff)

`[ON SCREEN]` Switch to the **MR tab (3)**; scroll to Shockwave's comment.

> **"Now put it in the pipeline. Open a merge request, and Shockwave's bot posts this review automatically — the risk verdict, the untested hotspots, the exact tests to run — computed live from Orbit. Intelligent orchestration, with context."**

2:10 – 2:40 · Five surfaces

`[ON SCREEN]` Briefly show the **agent page (tab 4)**, then the landing's "five surfaces".

> **"It's a CLI, this web monitor, the merge-request bot, a GitLab Duo agent in the AI Catalog, and a reusable Orbit skill — one graph, everywhere review happens."**

2:40 – 3:00 · Close

`[ON SCREEN]` Landing hero.

> **"Shockwave: see the blast radius, the exposure, and the tests you're missing — before you click merge. Review with context, not guesswork."**

Notes

- The hosted Blast Monitor analyzes real **snapshots** (`setupmethod` , `route` , `dispatch_request`) so it works with no backend. For **live analysis of any symbol**, run it locally: `pip install -e ".[web]"` → `orbit index <repo>` → `shockwave-web` .
- The **MR-bot comment is genuinely computed live** from Orbit Remote — that's your proof of real, end-to-end Orbit use.

After recording

- Upload to **YouTube or Vimeo** (unlisted is fine), keep it **≤ 3:00**.
- Title: *"Shockwave — know what a change breaks, before you merge (GitLab Orbit)"*.
- Paste the link into Devpost (write-up ready in `DEVPOST.md`).

Fallbacks

- A hosted seed doesn't render → use `setupmethod` (always baked); hard-refresh.
- Want a fully-live feel → record the local `shockwave-web` instead (any symbol, no snapshot note).